Numpy Crash Course

In [58]:
```
import numpy as np   #Numpy has 256 functions and for real time we mostly need 30 f
```

In [59]:
```
np.__version__
```

Out[59]:
```
'2.3.5'
```

In [60]:
```
my_list = [0,1,2,3,4,5]
my_list
```

Out[60]:
```
[0, 1, 2, 3, 4, 5]
```

In [61]:
```
type(my_list)
```

Out[61]:
```
list
```

In [62]:
```
arr = np.array(my_list) # this is used to convert the list to array
arr
```

Out[62]:
```
array([0, 1, 2, 3, 4, 5])
```

The Numpy arrays are mentioned in the above manner inside () and [] braces

In [63]:
```
type(arr)
```

Out[63]:
```
numpy.ndarray
```

In [70]:
```
arr
```

Out[70]:
```
array([0, 1, 2, 3, 4, 5])
```

In [71]:
```
np.arange(10)
```

Out[71]:
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [72]:
```
np.arange(10,19)
```

Out[72]:
```
array([10, 11, 12, 13, 14, 15, 16, 17, 18])
```

In [73]:
```
np.arange(10,20,3)
```

Out[73]:
```
array([10, 13, 16, 19])
```

In [ ]:
```
# paramter tuning means system given paramter
```

```
# hyper parameter means the user changes system paramter as per users requirement
```

In [74]:
```
np.zeros(5)  # we get 5 zeroes as system by default takes float
```

Out[74]:
```
array([0., 0., 0., 0., 0.])
```

In [75]:
```
np.zeros(5, dtype=int)  # to convert float to int we add dtype
```

Out[75]:
```
array([0, 0, 0, 0, 0])
```

In [76]:
```
np.zeros((2,3))  #it becomes 2d array
```

Out[76]:
```
array([[0., 0., 0.],
       [0., 0., 0.]])
```

In [77]:
```
np.zeros((2,3), dtype=int)
```

Out[77]:
```
array([[0, 0, 0],
       [0, 0, 0]])
```

In [ ]:
```
# we need to call the arrays as (2*2 matrix)
```

In [78]:
```
np.zeros((5,10), dtype=int)
```

Out[78]:
```
array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]])
```

In [ ]:
```
# Here the rows and columns need to counted from 0
```

In [79]:
```
np.ones(10)
```

Out[79]:
```
array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

In [80]:
```
np.ones((3,4), dtype=int)
```

Out[80]:
```
array([[1, 1, 1, 1],
       [1, 1, 1, 1],
       [1, 1, 1, 1]])
```

In [81]:
```
np.rand.rand(4)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[81], line 1
----> 1 np.rand.rand(4)

File ~\anaconda3\Lib\site-packages\numpy\__init__.py:795, in __getattr__(attr)
    792     import numpy.char as char
    793     return char.chararray
--> 795 raise AttributeError(f"module {__name__!r} has no attribute {attr!r}")

AttributeError: module 'numpy' has no attribute 'rand'
```

In [82]:
```
np.random.rand(4)  #it gives random numbers
```

Out[82]:
```
array([0.44500248, 0.4022525 , 0.39463355, 0.23974962])
```

In [86]:
```
np.random.rand(2,3)
```

Out[86]:
```
array([[0.95789601, 0.2928808 , 0.80622611],
       [0.68891295, 0.80711054, 0.14933357]])
```

In [87]:
```
np.random.randint(4)   # this is used to generate numbers for getting OTP in live s

#It also meane it gives random int numbers from 1 to 2 only
```

Out[87]:
```
2
```

In [90]:
```
np.random.randint(10,16)
```

Out[90]:
```
11
```

In [94]:
```
np.random.randint(1,5,3)
# In numpy (1,5,3) --1 is start, 5 is stop 3 is generate 3 random numbers
#In range we increment the values by 3
#it generates 3 random numbers between 1 to 5 excluding 5
# dtype 32 integer type meaning each element has 32 bit integer, depending on the s
```

Out[94]:
```
array([3, 2, 1], dtype=int32)
```

In [95]:
```
np.random.randint(8,3,2)   #since th first argument is grater than second argument
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[95], line 1
----> 1 np.random.randint(8,3,2)

File numpy/random/mtrand.pyx:794, in numpy.random.mtrand.RandomState.randint()

File numpy/random/_bounded_integers.pyx:1425, in numpy.random._bounded_integers._ran
d_int32()

ValueError: low >= high
```

In [98]:
```
np.random.randint(-8,3,2)
```

Out[98]:
```
array([-7,  1], dtype=int32)
```

In [101…
```
np.random.randint(10,40,(10,10))   #here 10*10 works as nd array and gives values
#it works as 10*10 matrix
```

Out[101…
```
array([[23, 38, 17, 22, 13, 27, 15, 19, 25, 22],
       [22, 22, 19, 29, 11, 23, 33, 17, 24, 22],
       [11, 14, 38, 20, 36, 21, 31, 38, 17, 16],
       [22, 13, 29, 36, 34, 18, 14, 25, 35, 37],
       [10, 21, 31, 26, 33, 21, 30, 16, 34, 16],
       [16, 26, 21, 35, 22, 23, 22, 10, 19, 19],
       [34, 17, 23, 37, 26, 31, 23, 24, 17, 24],
       [33, 21, 10, 12, 30, 38, 39, 23, 26, 20],
       [36, 11, 28, 33, 29, 25, 37, 23, 24, 38],
       [33, 20, 16, 22, 10, 30, 33, 13, 33, 26]], dtype=int32)
```

In [103…
```
ri = np.random.randint(10,40,(10,10))
ri
```

Out[103…
```
array([[13, 22, 29, 36, 27, 12, 23, 26, 37, 16],
       [26, 17, 25, 15, 35, 20, 31, 28, 22, 11],
       [15, 30, 22, 26, 39, 12, 37, 12, 12, 22],
       [20, 18, 16, 10, 10, 33, 32, 11, 30, 15],
       [15, 27, 20, 16, 19, 15, 33, 22, 11, 19],
       [30, 31, 34, 18, 27, 29, 39, 14, 13, 13],
       [37, 38, 13, 22, 20, 32, 11, 17, 18, 37],
       [23, 26, 24, 25, 38, 11, 19, 33, 32, 14],
       [39, 38, 30, 16, 19, 34, 35, 13, 22, 34],
       [35, 38, 18, 19, 11, 32, 11, 28, 10, 16]], dtype=int32)
```

In [104…
```
len(ri)  #it will print by default 10 rows and not columns
```

Out[104…
```
10
```

In [105…
```
ri.shape  #it gives rows and columns matrix
```

Out[105…
```
(10, 10)
```

In [106…
```
id(ri)  #it gives address of memory location
```

Out[106…
```
1636094871856
```

slicing in NUMPY

In [107…
```
ri[:]
```

Out[107…
```
array([[13, 22, 29, 36, 27, 12, 23, 26, 37, 16],
       [26, 17, 25, 15, 35, 20, 31, 28, 22, 11],
       [15, 30, 22, 26, 39, 12, 37, 12, 12, 22],
       [20, 18, 16, 10, 10, 33, 32, 11, 30, 15],
       [15, 27, 20, 16, 19, 15, 33, 22, 11, 19],
       [30, 31, 34, 18, 27, 29, 39, 14, 13, 13],
       [37, 38, 13, 22, 20, 32, 11, 17, 18, 37],
       [23, 26, 24, 25, 38, 11, 19, 33, 32, 14],
       [39, 38, 30, 16, 19, 34, 35, 13, 22, 34],
       [35, 38, 18, 19, 11, 32, 11, 28, 10, 16]], dtype=int32)
```

In [108…
```
ri[1]   #it generates first row full
```

Out[108…
```
array([26, 17, 25, 15, 35, 20, 31, 28, 22, 11], dtype=int32)
```

In [109…
```
ri[0:3]  #we get 3 rows (n-1)
```

Out[109…
```
array([[13, 22, 29, 36, 27, 12, 23, 26, 37, 16],
       [26, 17, 25, 15, 35, 20, 31, 28, 22, 11],
       [15, 30, 22, 26, 39, 12, 37, 12, 12, 22]], dtype=int32)
```

In [115…
```
ri_1 = np.random.randint(10,30,(10,10))
print(ri_1)  #if we call ri_1 using a print function, dtype=32 is not executed
```

```
[[22 29 29 11 10 14 17 24 10 28]
 [25 10 13 27 23 19 18 12 23 17]
 [12 10 26 12 29 24 29 20 24 21]
 [21 18 19 28 26 13 24 14 11 17]
 [15 19 26 11 17 28 22 10 17 19]
 [29 18 16 18 14 13 18 16 14 27]
 [12 22 25 11 17 10 24 18 12 29]
 [10 24 17 12 17 15 27 12 24 28]
 [15 21 13 15 18 18 21 27 18 14]
 [17 20 12 15 26 19 22 11 15 21]]
```

In [113…
```
ri_1.shape
```

Out[113…
```
(10, 10)
```

In [116…
```
ri_1[4:8]  #it executes rows from 4 to 7 .
# we need to count from 0 row to 5th row...where 5th row numbering is 4 only since
```

Out[116…
```
array([[15, 19, 26, 11, 17, 28, 22, 10, 17, 19],
       [29, 18, 16, 18, 14, 13, 18, 16, 14, 27],
       [12, 22, 25, 11, 17, 10, 24, 18, 12, 29],
       [10, 24, 17, 12, 17, 15, 27, 12, 24, 28]], dtype=int32)
```

In [117…
```
ri_1
```

Out[117…
```
array([[22, 29, 29, 11, 10, 14, 17, 24, 10, 28],
       [25, 10, 13, 27, 23, 19, 18, 12, 23, 17],
       [12, 10, 26, 12, 29, 24, 29, 20, 24, 21],
       [21, 18, 19, 28, 26, 13, 24, 14, 11, 17],
       [15, 19, 26, 11, 17, 28, 22, 10, 17, 19],
       [29, 18, 16, 18, 14, 13, 18, 16, 14, 27],
       [12, 22, 25, 11, 17, 10, 24, 18, 12, 29],
       [10, 24, 17, 12, 17, 15, 27, 12, 24, 28],
       [15, 21, 13, 15, 18, 18, 21, 27, 18, 14],
       [17, 20, 12, 15, 26, 19, 22, 11, 15, 21]], dtype=int32)
```

In [118…
```
ri_1[3,7]  #it starts from row 3 to the exact column number position which is colum
```

Out[118…
```
np.int32(14)
```

In [120…
```
print(ri_1[3,7])  # comma is used
```

```
14
```

In [122…
```
print(ri_1)
```

```
[[22 29 29 11 10 14 17 24 10 28]
 [25 10 13 27 23 19 18 12 23 17]
 [12 10 26 12 29 24 29 20 24 21]
 [21 18 19 28 26 13 24 14 11 17]
 [15 19 26 11 17 28 22 10 17 19]
 [29 18 16 18 14 13 18 16 14 27]
 [12 22 25 11 17 10 24 18 12 29]
 [10 24 17 12 17 15 27 12 24 28]
 [15 21 13 15 18 18 21 27 18 14]
 [17 20 12 15 26 19 22 11 15 21]]
```

In [123…
```
ri_1[-5]  #It calculates from bottom rows to 5th row and prints 5th row values
```

Out[123…
```
array([29, 18, 16, 18, 14, 13, 18, 16, 14, 27], dtype=int32)
```

In [125…
```
print(ri_1[-5,-3])  #it gives -5h row to -3 column value
```

```
16
```

In [126…
```
ri_1[::3]  #It will incerement every 3 rows print it
```

Out[126…
```
array([[22, 29, 29, 11, 10, 14, 17, 24, 10, 28],
       [21, 18, 19, 28, 26, 13, 24, 14, 11, 17],
       [12, 22, 25, 11, 17, 10, 24, 18, 12, 29],
       [17, 20, 12, 15, 26, 19, 22, 11, 15, 21]], dtype=int32)
```

In [127…
```
ri_1[::-1]  #printing whole array in reverse
```

Out[127…
```
array([[17, 20, 12, 15, 26, 19, 22, 11, 15, 21],
       [15, 21, 13, 15, 18, 18, 21, 27, 18, 14],
       [10, 24, 17, 12, 17, 15, 27, 12, 24, 28],
       [12, 22, 25, 11, 17, 10, 24, 18, 12, 29],
       [29, 18, 16, 18, 14, 13, 18, 16, 14, 27],
       [15, 19, 26, 11, 17, 28, 22, 10, 17, 19],
       [21, 18, 19, 28, 26, 13, 24, 14, 11, 17],
       [12, 10, 26, 12, 29, 24, 29, 20, 24, 21],
       [25, 10, 13, 27, 23, 19, 18, 12, 23, 17],
       [22, 29, 29, 11, 10, 14, 17, 24, 10, 28]], dtype=int32)
```

In [129…
```
ri_1[::-5]
```

Out[129…
```
array([[17, 20, 12, 15, 26, 19, 22, 11, 15, 21],
       [15, 19, 26, 11, 17, 28, 22, 10, 17, 19]], dtype=int32)
```

In [132…
```
print(ri_1[2])
print(ri_1[:,2])  #it prints all the column values from row 2
```

```
[12 10 26 12 29 24 29 20 24 21]
[29 13 26 19 26 16 25 17 13 12]
```

In [133…
```
ri_1[:,8]
```

Out[133…
```
array([10, 23, 24, 11, 17, 14, 12, 24, 18, 15], dtype=int32)
```

In [134…
```
ri_1[:,-8]
```

Out[134…
```
array([29, 13, 26, 19, 26, 16, 25, 17, 13, 12], dtype=int32)
```

In [135…
```
ri_1
```

Out[135…
```
array([[22, 29, 29, 11, 10, 14, 17, 24, 10, 28],
       [25, 10, 13, 27, 23, 19, 18, 12, 23, 17],
       [12, 10, 26, 12, 29, 24, 29, 20, 24, 21],
       [21, 18, 19, 28, 26, 13, 24, 14, 11, 17],
       [15, 19, 26, 11, 17, 28, 22, 10, 17, 19],
       [29, 18, 16, 18, 14, 13, 18, 16, 14, 27],
       [12, 22, 25, 11, 17, 10, 24, 18, 12, 29],
       [10, 24, 17, 12, 17, 15, 27, 12, 24, 28],
       [15, 21, 13, 15, 18, 18, 21, 27, 18, 14],
       [17, 20, 12, 15, 26, 19, 22, 11, 15, 21]], dtype=int32)
```

In [136…
```
ri_1[2:5,3:7]
```

Out[136…
```
array([[12, 29, 24, 29],
       [28, 26, 13, 24],
       [11, 17, 28, 22]], dtype=int32)
```

In [137…
```
ri_1[0:3,0:3]
```

Out[137…
```
array([[22, 29, 29],
       [25, 10, 13],
       [12, 10, 26]], dtype=int32)
```

In [138…
```
np.arange(1,13)
```

Out[138…
```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12])
```

In [140…
```
np.arange(1,13).reshape(3,4)  # it divides and sperates into 3rows and 4 columns
```

Out[140…
```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

In [141…
```
np.arange(1,13).reshape(5,4)  #exceeded values cant create matrix
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[141], line 1
----> 1 np.arange(1,13).reshape(5,4)

ValueError: cannot reshape array of size 12 into shape (5,4)
```

In [143…
```
np.arange(1,13).reshape(12,1)
```

Out[143…
```
array([[ 1],
       [ 2],
       [ 3],
       [ 4],
       [ 5],
       [ 6],
       [ 7],
       [ 8],
       [ 9],
       [10],
       [11],
       [12]])
```

In [ ]:
```
# Numpy Masking or Filter
```

In [146…
```
mat = np.arange(0,100).reshape(10,10)
mat
```

Out[146…
```
array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
       [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
       [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
       [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
       [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
       [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],
       [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],
       [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
```

In [147…
```
mat < 50  #it just checks condition and gives true or false details
```

Out[147…
```
array([[ True,  True,  True,  True,  True,  True,  True,  True,  True,
         True],
       [ True,  True,  True,  True,  True,  True,  True,  True,  True,
         True],
       [ True,  True,  True,  True,  True,  True,  True,  True,  True,
         True],
       [ True,  True,  True,  True,  True,  True,  True,  True,  True,
         True],
       [ True,  True,  True,  True,  True,  True,  True,  True,  True,
         True],
       [False, False, False, False, False, False, False, False, False,
        False],
       [False, False, False, False, False, False, False, False, False,
        False],
       [False, False, False, False, False, False, False, False, False,
        False],
       [False, False, False, False, False, False, False, False, False,
        False],
       [False, False, False, False, False, False, False, False, False,
        False]])
```

In [148…
```
mat[mat<50]  #when u use mat twice then u will get the values upto 49
#This concept is known as masking or filter
```

Out[148…
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
       34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49])
```

In [149…
```
mat[mat<=50]
```

Out[149…
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
       34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50])
```

In [150…
```
mat[mat==50]  #it prints exact 50 value only
```

Out[150…
```
array([50])
```

In [151…
```
mat[mat!=50] #it prints other than 50
```

Out[151…
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
       34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 51,
       52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68,
       69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85,
       86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])
```

In [152…
```
mat[mat>50]
```

Out[152…
```
array([51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67,
       68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84,
       85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: